PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Arquitectura Web y APIs REST con Python (Flask).
- **Competencia:** Construcción de software y exposición de servicios web.
- **Resultados de Aprendizaje:** 1. Comprender el protocolo HTTP y la arquitectura Cliente-Servidor. 2. Implementar un servidor web utilizando el micro-framework Flask bajo el estándar WSGI. 3. Serializar estructuras de datos a formato JSON y procesar Payloads entrantes (GET/POST). 4. Implementar políticas de seguridad perimetral (CORS) y aislar configuraciones mediante Variables de Entorno. 5. Acoplar la lógica de negocio Orientada a Objetos a una capa de transporte web (Arquitectura de Capas). 6. Validar y testear servicios web utilizando herramientas de la industria (Postman/Thunder Client).
- **Duración de la Guía:** 80 horas.

2. PRESENTACIÓN

De la Consola a la Nube En los Niveles 1 y 2 aprendiste a dominar la memoria RAM de tu computadora. Creaste variables, estructuraste diccionarios, modularizaste con funciones y modelaste el mundo real con Programación Orientada a Objetos (POO). Pero hasta hoy, tu software está aislado; solo tú puedes usarlo desde tu terminal.

El software moderno (como los ecosistemas educativos o agroindustriales escalables) no funciona así. Hoy, una aplicación móvil (Frontend) necesita comunicarse con un servidor central (Backend) para consultar o modificar información. ¿Cómo hablan dos sistemas distintos? A través de la Web.

En esta guía daremos el salto arquitectónico definitivo. Aprenderás a usar **Flask** no como "magia", sino como una Capa de Transporte que recibe mensajes de internet (HTTP), los traduce para que tus Clases de Python los entiendan, y devuelve respuestas estructuradas (JSON). Además, configuraremos nuestro servidor para que esté listo para producción, implementando políticas de seguridad (CORS) y ocultando datos sensibles, abandonando el navegador como única herramienta de pruebas para adoptar **Postman** como verdaderos ingenieros.

3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

FASE 1: Abriendo los puertos (El Servidor Web y el estándar WSGI)

Concepto: Tu computadora tiene 65.535 "puertos" de red. Un servidor web es un proceso en la memoria RAM que implementa un bucle infinito escuchando permanentemente un puerto específico (ej. 5000), esperando a que un cliente establezca una conexión TCP.

Explicación Detallada (El Por Qué Técnico): Flask utiliza una librería interna llamada `Werkzeug` que implementa el estándar **WSGI** (Web Server Gateway Interface). Su trabajo es tomar el flujo de bytes que llega por el cable de red (la petición HTTP) y convertirlo en un Objeto de Python. Cuando hacemos `app = Flask(__name__)`, instanciamos la clase principal que orquestará este mapeo de red a memoria RAM.

Ejemplo Guiado 1.1: Nuestro primer servidor (`01_server.py`)

```
# 1. Importamos la Clase principal del framework
from flask import Flask

# 2. INSTANCIACIÓN: Creamos el objeto servidor.
# __name__ le dice a Flask dónde buscar los archivos internos del proyecto.
app = Flask(__name__)

# 3. ENRUTAMIENTO: El decorador '@' mapea una URL de red con una función en RAM.
@app.route('/', methods=['GET'])
def home() -> str:
    """Función que se ejecuta cuando el cliente hace GET a la raíz del sitio."""
    return "<h1>Sistema de Control Agroindustrial Activo</h1>"

# 4. EJECUCIÓN DEL BUCLE DE ESCUCHA
if __name__ == '__main__':
    # host='0.0.0.0' expone el servidor a todas las interfaces de tu red local (L
    # port=5000 es el puerto TCP asignado. debug=True reinicia el server si hay c
    print("Iniciando servidor en [http://127.0.0.1:5000](http://127.0.0.1:5000)")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Ejercicios Prácticos Fase 1:

1. Instala Flask en tu entorno virtual: `pip install flask`.
2. Ejecuta `01_server.py`.
3. Abre tu navegador e ingresa a `http://localhost:5000`. Luego, desde tu celular conectado a la misma red WiFi, ingresa a `http://[TU_IP_LOCAL]:5000` para comprobar la exposición del puerto.

FASE 2: Inyección Dinámica y Extracción de Datos (Rutas)

Concepto: Un sistema web real maneja múltiples "Endpoints" (Puntos de terminación de la API) para distintos recursos (`/usuarios`, `/inventario`). Además, la propia URL funciona como un

vehículo de transporte de parámetros.

Explicación Detallada (El Por Qué Técnico): Cuando escribimos `<tipo:variable>` en el decorador de la ruta, el Router de Flask aplica expresiones regulares para interceptar ese segmento de la URL HTTP, lo extrae, lo convierte dinámicamente al tipo de dato en Python (ej. un `string` de la URL se vuelve un `int` en la RAM) y lo inyecta como argumento en la función subyacente.

Ejemplo Guiado 2.1: Rutas Dinámicas y Tipadas (`02_rutas.py`)

```
from flask import Flask

app = Flask(__name__)

# Ruta DINÁMICA: Extraemos un texto (String) de la URL
@app.route('/api/saludar/<string:nombre>', methods=['GET'])
def saludar(nombre: str) -> str:
    return f"Hola {nombre}, tu solicitud fue enrutada exitosamente."

# Ruta DINÁMICA: Extraemos un entero (Integer) de la URL
@app.route('/api/calcular_iva/<int:precio>', methods=['GET'])
def calcular(precio: int) -> str:
    iva: float = precio * 0.19
    total: float = precio + iva
    return f"Precio: ${precio} | IVA: ${iva} | Total: ${total}"

if __name__ == '__main__':
    app.run(debug=True)
```

Ejercicios Prácticos Fase 2:

1. 🎯 **[EVIDENCIA DE DESEMPEÑO 1] - Endpoint Matemático:** Crea un endpoint `@app.route('/api/temperatura/<float:grados_c>')` que reciba una temperatura en grados Celsius desde la URL, la convierta internamente a Fahrenheit `(C * 9/5) + 32`, y retorne un texto con el resultado. *El instructor validará en vivo el manejo de la URL y la conversión estricta de tipos.*

FASE 3: El Estándar Universal de Intercambio (APIs REST y JSON)

Concepto: Retornar HTML `<h1>` es útil para páginas estáticas, pero el software moderno requiere desacoplamiento. El Backend y el Frontend deben ser proyectos separados que se comunican intercambiando "Datos Crudos". El formato rey de internet es **JSON**, que equivale a los **Diccionarios** de Python.

Explicación Detallada (El Por Qué Técnico): Una app móvil en Flutter no entiende qué es una "Clase" o una "Lista" de Python. La función `jsonify` de Flask toma tus estructuras de datos en

la memoria RAM y las serializa (las aplana a una cadena de texto estructurada). Además, inyecta la cabecera HTTP crítica `Content-Type: application/json` para que el cliente receptor sepa cómo parsear los bytes entrantes.

Ejemplo Guiado 3.1: Exponiendo la RAM como API (`03_json.py`)

```
from flask import Flask, jsonify, Response

app = Flask(__name__)

# Base de datos simulada en memoria (Lista de Diccionarios)
sensores_trapiche: list[dict] = [
    {"id": "S1", "tipo": "Temperatura", "valor": 95.5, "estado": "Activo"},
    {"id": "S2", "tipo": "Presion", "valor": 40.2, "estado": "Alerta"}
]

@app.route('/api/sensores', methods=['GET'])
def obtener_sensores() -> Response:
    """Retorna todo el inventario serializado."""
    return jsonify({
        "total_registros": len(sensores_trapiche),
        "data": sensores_trapiche
    })

@app.route('/api/sensores/<string:id_sensor>', methods=['GET'])
def buscar_sensor(id_sensor: str) -> tuple[Response, int]:
    """Búsqueda específica. Retorna el objeto y un Código de Estado HTTP."""
    for sensor in sensores_trapiche:
        if sensor["id"] == id_sensor:
            return jsonify(sensor), 200 # 200 OK

    # Arquitectura resiliente: 404 (Not Found)
    return jsonify({"error": "Sensor no encontrado"}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

FASE 4: Inyección de Payloads y Testeo de Industria (POST y Postman)

Concepto: Un API de solo lectura (`GET`) está incompleta. Para insertar información, el cliente debe enviar un paquete de datos oculto en la petición HTTP llamado **Payload**, utilizando el método `POST` .

Explicación Detallada (El Por Qué Técnico): A diferencia de las variables en la URL, un Payload JSON viaja en el "Cuerpo" (Body) de la petición HTTP. Para leerlo, Flask expone el objeto global `request` . Llamar a `request.get_json()` captura los bytes de la red, verifica su integridad estructural, y los convierte automáticamente en un Diccionario en RAM. *Nota de Arquitectura:* Los navegadores web por defecto solo hacen peticiones `GET` . Para simular peticiones `POST`

complejas, es obligatorio abandonar el navegador y utilizar herramientas de la industria como **Postman**.

Ejemplo Guiado 4.1: Recibiendo JSON del cliente (`04_post.py`)

```
from flask import Flask, request, jsonify, Response

app = Flask(__name__)
usuarios_db: list[dict] = []

@app.route('/api/usuarios', methods=['POST'])
def crear_usuario() -> tuple[Response, int]:
    # 1. Extraemos el Payload del Body de la petición HTTP
    datos_entrantes: dict = request.get_json()

    # 2. Resiliencia: Validación del contrato de datos
    if not datos_entrantes or "nombre" not in datos_entrantes:
        return jsonify({"error": "Falta el campo requerido 'nombre'"}), 400 # 400

    # 3. Procesamiento y guardado
    nuevo_usuario = {
        "id": len(usuarios_db) + 1,
        "nombre": datos_entrantes["nombre"],
        "rol": datos_entrantes.get("rol", "Usuario Estándar")
    }
    usuarios_db.append(nuevo_usuario)

    # 4. Retorno de confirmación con código 201 (Created)
    return jsonify({"mensaje": "Usuario creado", "data": nuevo_usuario}), 201

if __name__ == '__main__':
    app.run(debug=True)
```

Ejercicios Prácticos Fase 4:

1. Instala Postman o la extensión Thunder Client en tu editor.
2. Levanta el servidor `04_post.py` .
3. Crea una petición `POST` hacia `http://localhost:5000/api/usuarios` . En "Body", selecciona "raw" y "JSON". Inyecta `{"nombre": "Carlos", "rol": "Admin"}` . Verifica que el servidor responda con código 201.

FASE 5: Preparación para Producción (CORS y Variables de Entorno)

Concepto: El código ya funciona, pero si intentas conectarlo a un Frontend web (ej. React) alojado en otro puerto, el navegador bloqueará la conexión por seguridad. Además, si subes tu código a GitHub con credenciales visibles, pondrás en riesgo tu infraestructura.

Explicación Detallada (El Por Qué Técnico): 1. **CORS (Cross-Origin Resource Sharing):** Es un mecanismo de seguridad de los navegadores web. Impide que un sitio web malicioso en `dominio-a.com` haga peticiones ocultas a tu API en `dominio-b.com`. Para permitir conexiones legítimas de tu propio Frontend, debes indicarle a Flask que inyecte la cabecera `Access-Control-Allow-Origin` en sus respuestas. 2. **Variables de Entorno (`.env`):** Quemar (hardcodear) claves de bases de datos o el estado de `debug=True` en el código fuente es el error de seguridad #1 en desarrollo. Las variables cambian según el entorno (Desarrollo local vs. Producción Cloud) y deben aislarse en un archivo invisible que el sistema operativo lee al arrancar.

Ejemplo Guiado 5.1: Configuración de Entorno Profesional (`05_produccion.py`)

```
# Requiere instalación previa: pip install flask-cors python-dotenv
import os
from flask import Flask, jsonify
from flask_cors import CORS
from dotenv import load_dotenv

# 1. Cargar variables desde el archivo .env hacia el entorno del S.O.
load_dotenv()

app = Flask(__name__)
# 2. Aplicar CORS a toda la aplicación (Permite que Frontends se conecten a la AP
CORS(app)

@app.route('/api/config', methods=['GET'])
def obtener_config():
    # 3. Leer variables de entorno (Nunca subir el .env a Git)
    ambiente_actual = os.getenv("FLASK_ENV", "Producción")
    puerto_asignado = os.getenv("PORT", 5000)

    return jsonify({
        "status": "Servidor seguro en línea",
        "entorno": ambiente_actual,
        "puerto": puerto_asignado
    }), 200

if __name__ == '__main__':
    # La configuración dinámica protege contra despliegues accidentales
    debug_mode = os.getenv("FLASK_DEBUG", "False").lower() == "true"
    port = int(os.getenv("PORT", 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

(Debes crear un archivo llamado `.env` en la misma carpeta con este contenido:)

```
FLASK_ENV=Desarrollo
FLASK_DEBUG=True
```

```
PORT=5000
```

FASE 6: La Cúspide (Integración POO, Flask y Arquitectura de Capas)

Concepto: En un diseño limpio, Flask *no debe* tener lógica matemática compleja en sus enrutadores. El archivo `app.py` solo hace de interfaz de red (recibe HTTP, entrega HTTP). El procesamiento pesado debe delegarse a Clases de Python aisladas. A esto se le llama **Separación de Responsabilidades**.

Ejemplo Guiado 6.1: API de Inventario (`06_arquitectura.py`)

```
import os
from flask import Flask, jsonify, request, Response
from flask cors import CORS
from dotenv import load_dotenv

load_dotenv()
app = Flask(__name__)
CORS(app)

# --- CAPA DE LÓGICA DE NEGOCIO (Core / POO) ---
class InventarioTrapiche:
    def __init__(self):
        self._productos: list[dict] = []

    def agregar_lote(self, tipo: str, kilos: float) -> dict:
        lote = {"id": len(self._productos)+1, "tipo": tipo, "kilos": kilos}
        self._productos.append(lote)
        return lote

    def obtener_todos(self) -> list[dict]:
        return self._productos

# Instanciación del Singleton (Gestor único de estado en RAM)
gestor_inventario = InventarioTrapiche()

# --- CAPA DE TRANSPORTE (Controladores / Flask) ---
@app.route('/api/inventario', methods=['GET'])
def ver_inventario() -> tuple[Response, int]:
    # El Controlador web delega la tarea a la Clase de Negocio
    datos = gestor_inventario.obtener_todos()
    return jsonify({"total": len(datos), "lotes": datos}), 200

@app.route('/api/inventario', methods=['POST'])
def registrar_lote() -> tuple[Response, int]:
    try:
        payload = request.get_json()
        # Delegamos la lógica de creación al Objeto
        nuevo_lote = gestor_inventario.agregar_lote(payload["tipo"], payload["kil
        return jsonify({"mensaje": "Lote registrado", "data": nuevo_lote}), 201
    except KeyError:
```

```
        return jsonify({"error": "Estructura JSON inválida."}), 400

if __name__ == '__main__':
    app.run(port=int(os.getenv("PORT", 5000)), debug=os.getenv("FLASK_DEBUG") ==
```

Ejercicios Prácticos Fase 6:

1. 🎯 **[EVIDENCIA DE PRODUCTO FINAL] - API Empresarial "Ready for Prod":** Crea un script `07_producto_api.py`.
    - Implementa una clase `Producto` con atributos encapsulados y métodos transaccionales.
    - Crea la Capa de Transporte en Flask con CORS habilitado y configuración atada a un archivo `.env`.
    - Implementa un CRUD básico (GET para listar, POST leyendo Payload JSON para crear).
    - **Requisito Postman:** Debes crear una "Colección" en Postman con al menos dos peticiones guardadas (GET y POST) y tomar capturas de pantalla de los resultados (Status 200 y 201).
    - Sube tu código (SIN el archivo `.env`) y las capturas de Postman a Zajuna.

4. PREPARACIÓN PARA LA EVIDENCIA DE CONOCIMIENTO

Para el cuestionario técnico en el LMS Zajuna, domina los siguientes conceptos arquitectónicos:

1. **Protocolo HTTP y Verbos:** Diferenciar estrictamente la intención de una petición `GET` (Idempotente, solo lectura) de una petición `POST` (Envío de un *Payload* para alterar el estado del servidor).
2. **Códigos de Estado HTTP:** Comprender qué significan a nivel de ingeniería las respuestas: `200 OK`, `201 Created`, `400 Bad Request` (Error del cliente por JSON mal formado), `404 Not Found` y `500 Internal Server Error` (Crash no manejado en código).
3. **CORS y Entornos:** Explicar por qué CORS bloquea las peticiones de navegadores cruzados (Cross-Origin) y por qué las credenciales de bases de datos o puertos jamás deben escribirse en código plano (`.env`).
4. **Arquitectura de Capas:** Entender por qué se debe delegar el trabajo matemático a una Clase (POO) y dejar a Flask únicamente como enrutador de cabeceras y bytes (Separación de Responsabilidades).

5. GLOSARIO TÉCNICO

- **CORS (Cross-Origin Resource Sharing):** Mecanismo que utiliza cabeceras HTTP adicionales para permitir que un cliente (User Agent/Navegador) obtenga permiso para

acceder a recursos de un servidor en un origen distinto al suyo propio.

- **Variables de Entorno (.env):** Variables dinámicas inyectadas al nivel del Sistema Operativo que afectan el comportamiento de los procesos en ejecución, vitales para ocultar claves ('secrets').
- **API REST:** Interfaz que permite que dos sistemas informáticos independientes se comuniquen a través de HTTP, intercambiando datos puros en formato JSON.
- **Payload:** La "carga útil" o los datos reales que viajan inyectados en el cuerpo (Body) de una petición `POST` o `PUT`.
- **Postman / Thunder Client:** Clientes HTTP utilizados por ingenieros de Backend y QA para probar, simular y automatizar peticiones complejas sin necesidad de programar una interfaz gráfica Frontend.
- **WSGI (Web Server Gateway Interface):** Estándar universal de Python que especifica cómo el servidor de red se comunica con el framework de desarrollo de la aplicación web.

6. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
Ejecución	Codificar los módulos del sistema de información e interfaces.	**AP07 - Implementación de Servicios Web.**	**1. Evidencia de Conocimiento:**	• Identifica la arquitectura Cliente-Servidor en la Web.	**Técnica:** Formulación de preguntas.
		Asimilar los conceptos de protocolo HTTP, servidores web WSGI (Flask), CORS y serialización de datos (JSON).	Cuestionario técnico sobre la arquitectura Cliente-Servidor, diferencias entre verbos HTTP, códigos de estado (200, 201, 400, 404, 500), políticas CORS y gestión de entornos.	• Diferencia un Diccionario de memoria RAM de un formato serializado JSON. • Reconoce la necesidad de abstraer variables sensibles del código fuente.	**Instrumento:** Cuestionario (LMS Zajuna).
Ejecución	Codificar los módulos	**AP08 - Enrutamiento y**	**2. Evidencia de Desempeño:**	• Codifica rutas web utilizando	**Técnica:** Observación

	del sistema de información e interfaces.	**Exposición de Datos.**		el micro-framework Flask.	Sistemática y Code Review.
		Desarrollar Endpoints web tipados para el intercambio dinámico de información.	Participación y observación directa del aprendiz codificando en vivo el **Endpoint Matemático** (Fase 2) comprobando la inyección dinámica de variables de red.	• Gestiona variables inyectadas desde la petición HTTP hacia la lógica interna de Python mediante *Path variables* o *Query strings*.	**Instrumento:** Rúbrica de Desempeño Técnico.
Ejecución	Codificar los módulos del sistema de información e interfaces.	**AP09 - Integración Backend (POO, APIs y Testing).**	**3. Evidencia de Producto:**	• Desarrolla una API REST 100% funcional orientada a la arquitectura de capas.	**Técnica:** Valoración de Producto.
		Integrar métodos de negocio a través de una API REST protegida y evaluada con herramientas de industria.	Entrega de archivo (.zip) alojado en Zajuna conteniendo: 1. El script `app.py` del **Reto Final Integrador** (Con CORS y llamadas al .env). 2. Capturas evidenciando la validación de peticiones GET/POST mediante **Postman**.	• Aplica políticas CORS y abstracción de entorno. • Demuestra capacidad para inyectar *Payloads* (JSON) mediante clientes HTTP especializados.	**Instrumento:** Lista de Verificación (Inspección de Endpoints y Capturas de Postman).